Design and Analysis of Algorithms

BCS-503 · AKTU Exam Answers · Topper answer-sheet style

> **How to use these notes:** DAA papers mostly ask for **traces**. Always write the **algorithm or formula first**, then a **step table**, then the **final answer in a box**. Draw every tree or graph.

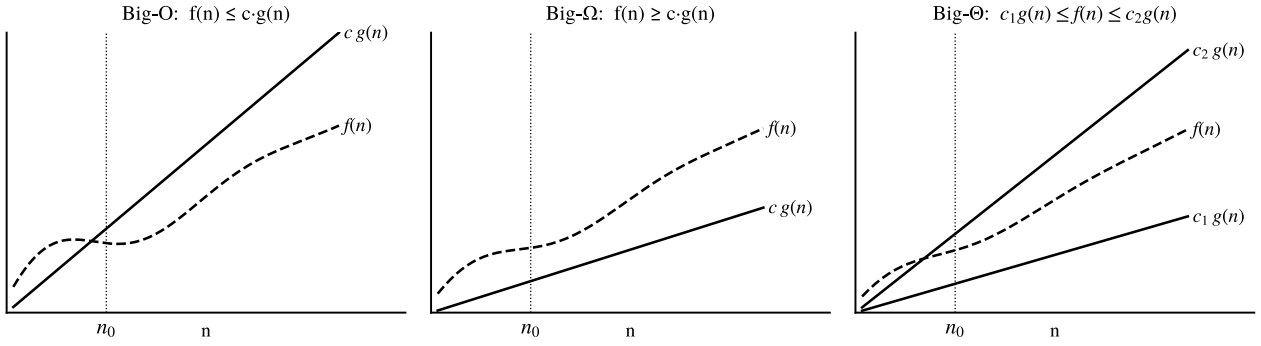
UNIT 1: COMPLEXITY, RECURRENCES & SORTING

Section A: 2-Mark Questions

Q1. Define Big-O, Big-Omega and Big-Theta. *[2 Marks]*

Ans.

- **Big-O (upper bound):** $f(n) = O(g(n))$ if $0 \le f(n) \le c\,g(n)$ for all $n \ge n_0$.
- **Big-Ω (lower bound):** $f(n) = \Omega(g(n))$ if $0 \le c\,g(n) \le f(n)$ for all $n \ge n_0$.
- **Big-Θ (tight bound):** $c_1 g(n) \le f(n) \le c_2 g(n)$ for all $n \ge n_0$.
- Example: $3n + 2 = \Theta(n)$.



Asymptotic bounds: O gives an upper bound, Ω a lower bound and Θ a tight bound (for all $n \ge n_0$)

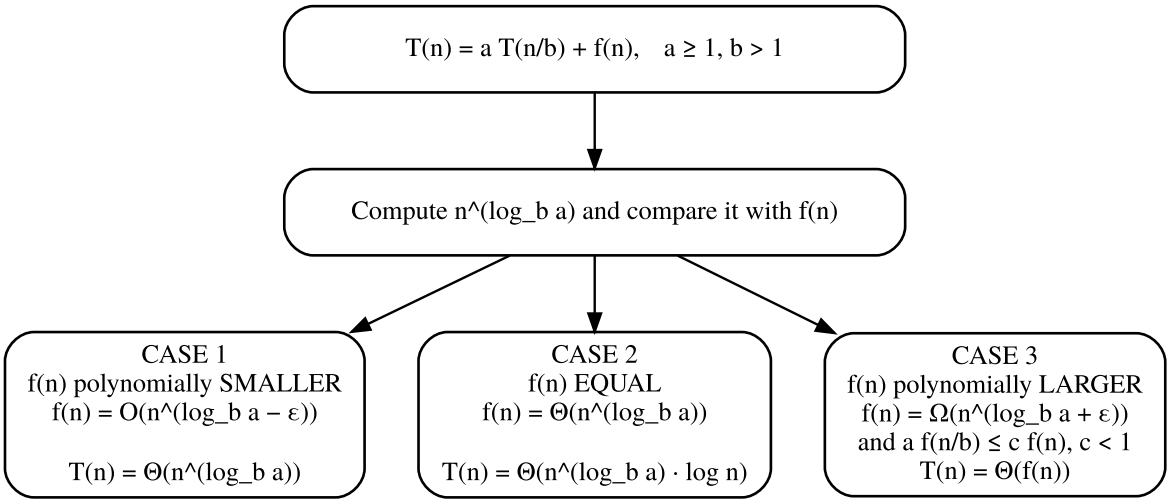
Q2. Stable vs. unstable sorting. *[2 Marks]*

Ans. A **stable** sort keeps the **relative order of equal keys** (for example 5a before 5b). **Stable:** Bubble, Insertion, **Merge**, Counting, Radix. **Unstable: Quick**, **Heap**, Selection.

Section B: 7-Mark Questions

Q3. Solve T(n) = 2T(n/2) + n and T(n) = 3T(n/9) + n. *[7 Marks]*

Ans. Master Theorem: for $T(n) = aT(n/b) + f(n)$, compare $f(n)$ with $n^{\log_b a}$.



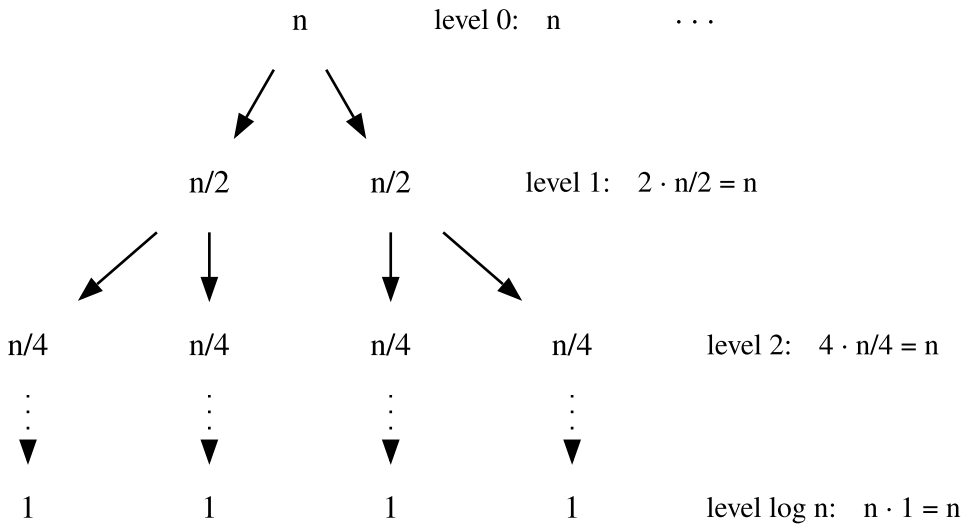
Master Theorem: compare f(n) with n^(log_b a)

(i) T(n) = 2T(n/2) + n

- a = 2, b = 2, so $n^{\log_2 2} = n$, and $f(n) = n$. They are equal, so this is **Case 2**.

$$T(n) = \Theta(n \log n)$$

Recursion tree check: every level costs n, and there are $\log_2 n + 1$ levels.



Recursion tree for T(n) = 2T(n/2) + n: every level costs n, and there are $\log_2 n + 1$ levels

(ii) T(n) = 3T(n/9) + n

- a = 3, b = 9, so $n^{\log_9 3} = n^{1/2}$. Since $f(n) = n = \Omega\left(n^{1/2+\varepsilon}\right)$, f(n) is larger, so check **Case 3**.
- **Regularity:** $3f(n/9) = n/3 \le cn$ with $c = 1/3 < 1$ ✓

$$T(n) = \Theta(n)$$

Recursion tree check: the level costs are $n + n/3 + n/9 + \cdots \le 1.5n$.

> **Exam Tip:** the Master Theorem does not apply to $T(n) = T(n-1) + n$ or to the "gap" cases.

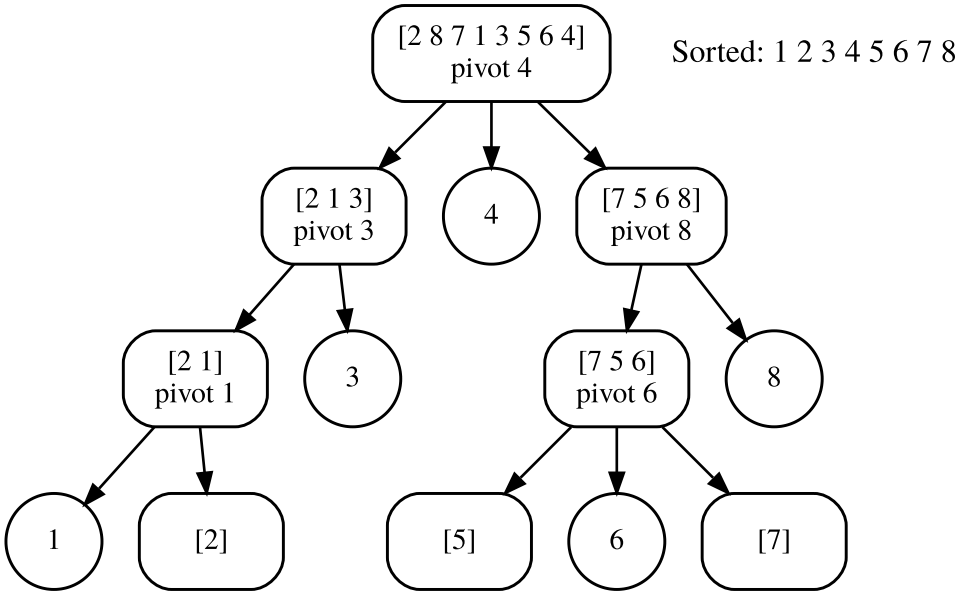
Q4. Explain Quick Sort, trace it, and say why it is preferred over Merge Sort. *[7 Marks]*

Ans. Quick Sort (Hoare) is a **divide-and-conquer, in-place** sort. It **partitions** the array around a **pivot** and then sorts both parts recursively.

```
PARTITION(A, p, r):  x = A[r]; i = p - 1
   for j = p to r - 1:  if A[j] <= x:  i = i + 1; swap A[i], A[j]
   swap A[i + 1], A[r];  return i + 1
```

Trace on A = [2, 8, 7, 1, 3, 5, 6, 4] (pivot = 4)

j	A[j]	≤ 4?	i	Array
1	2	Yes	1	2 8 7 1 3 5 6 4
4	1	Yes	2	2 1 7 8 3 5 6 4
5	3	Yes	3	2 1 3 8 7 5 6 4
end	swap A4, A8			2 1 3 **[4]** 7 5 6 8



Recursive calls of Quick Sort on [2, 8, 7, 1, 3, 5, 6, 4] (circled = pivot placed in its final position)

Complexity:

- Best and average: $\Theta(n \log n)$.
- **Worst** (sorted input): $\Theta(n^2)$.
- Space: $O(\log n)$.

Why it is preferred over Merge Sort:

1. **In-place:** no O(n) extra array is needed.
2. **Better cache locality.**
3. **Smaller constant factors.**
4. With a **randomized pivot**, the worst case is rare.

Quick Sort	Merge Sort
Worst case $O(n^2)$	Always $O(n \log n)$
$O(\log n)$ space	$O(n)$ space
Not stable	Stable

Conclusion: used in C's `qsort` and Java's primitive-array sort.

Q5. Trace Counting Sort and Heap Sort. *[7 Marks]*

Ans.

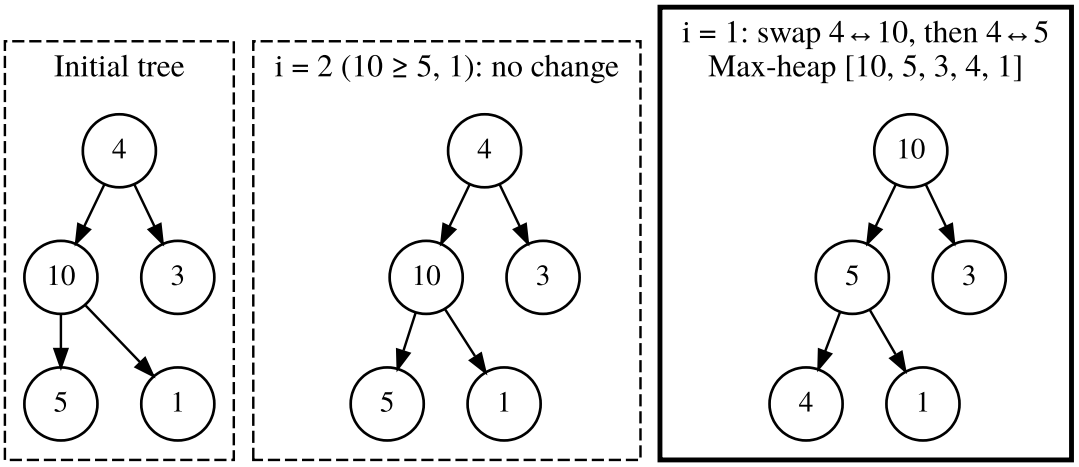
(a) Counting Sort on A = [2, 5, 3, 0, 2, 3, 0, 3], k = 5. This is a **non-comparison, stable** sort that runs in $O(n + k)$.

i	0	1	2	3	4	5
Count C[i]	2	0	2	3	0	1
Cumulative C[i]	2	2	4	7	7	8

Scan A from the **right**: put $B[C[A[j]]] = A[j]$, then decrease C. For example A[8] = 3 goes to B[7], and A[7] = 0 goes to B[2].

$$B = [0, 0, 2, 2, 3, 3, 3, 5]$$

(b) Heap Sort on A = [4, 10, 3, 5, 1]. **Build a max-heap**, then repeatedly swap the root with the last element and heapify. It runs in $O(n \log n)$, in place.



BUILD-MAX-HEAP on [4, 10, 3, 5, 1]

Pass	Swap	Heap after heapify / sorted part
1	10 ↔ 1	[5, 4, 3, 1 / 10]
2	5 ↔ 1	[4, 1, 3 / 5, 10]
3	4 ↔ 3	[3, 1 / 4, 5, 10]
4	3 ↔ 1	[1 / 3, 4, 5, 10]

Sorted: $[1, 3, 4, 5, 10]$

UNIT 2: ADVANCED DATA STRUCTURES

Section A: 2-Mark Questions

Q1. Properties of a Red-Black Tree. *[2 Marks]*

Ans.

1. Every node is **red or black**.
2. The **root is black**.
3. Every **leaf (NIL) is black**.
4. A **red node has black children**: no two reds in a row.
5. **Every path from a node to its leaves has the same number of black nodes** (the black-height).

So the height is at most $2 \log_2 (n + 1)$.

Q2. Define a B-Tree of order m. *[2 Marks]*

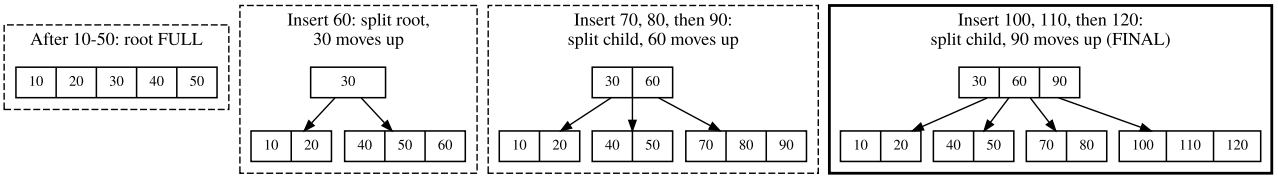
Ans. A balanced m-way search tree in which:

- Each node has at most **m children and m − 1 keys**.
- Each non-root node has at least **⌈m/2⌉ children**.
- The root has at least 2 children.
- **All leaves are at the same level.**
- Keys in each node are sorted.

Section B: 7-Mark Questions

Q3. Insert keys into an empty B-Tree of minimum degree t = 3. *[7 Marks]*

Ans. With **t = 3**, each node holds **2 to 5 keys** (the root 1 to 5). A **full node (5 keys) is split**, and its **median moves up**. CLRS splits full nodes **on the way down**.



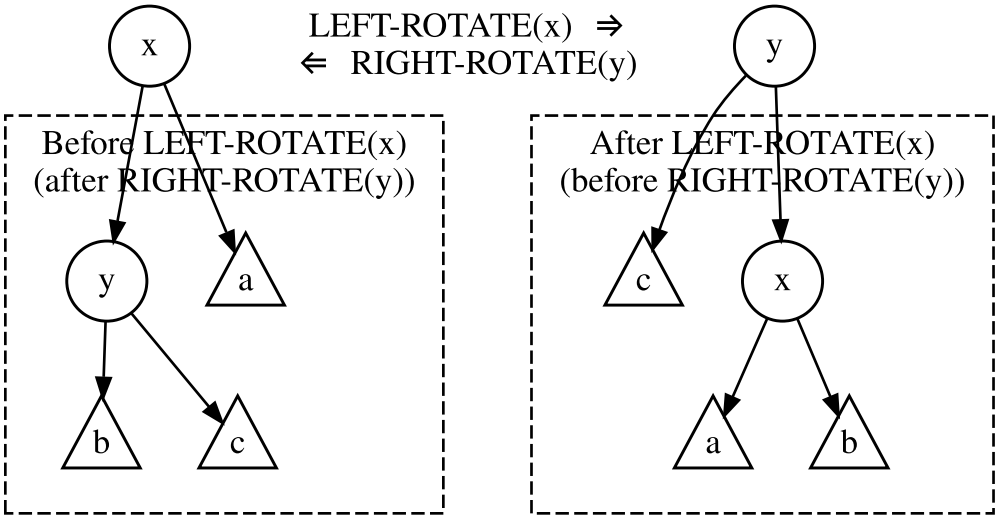
B-Tree (t = 3, max 5 keys per node): inserting 10, 20, ..., 120

> **Exam Tip:** for t = 3: max keys = 2t − 1 = 5, min keys = t − 1 = 2, max children = 2t = 6.

Conclusion: B-Trees keep **all leaves at the same level**, giving $O(\log n)$ disk accesses. They are used in DBMS and file systems.

Q4. Show insertions and rotations in a Red-Black Tree. *[7 Marks]*

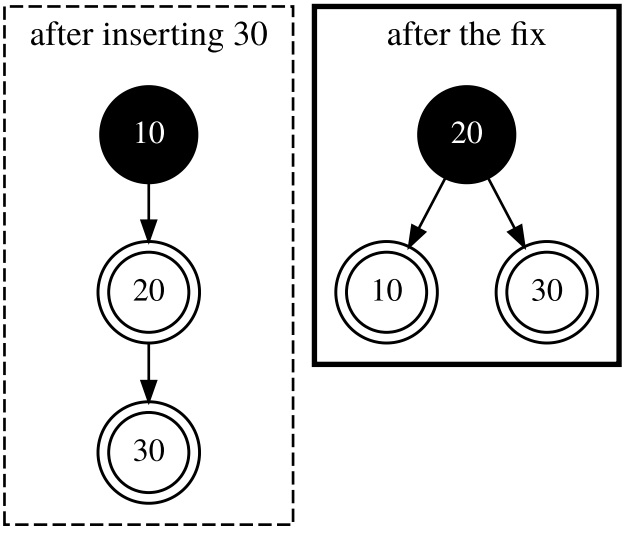
Ans. A new node is inserted as **RED**. If its parent is also red, fix it with **recolouring or rotations**.



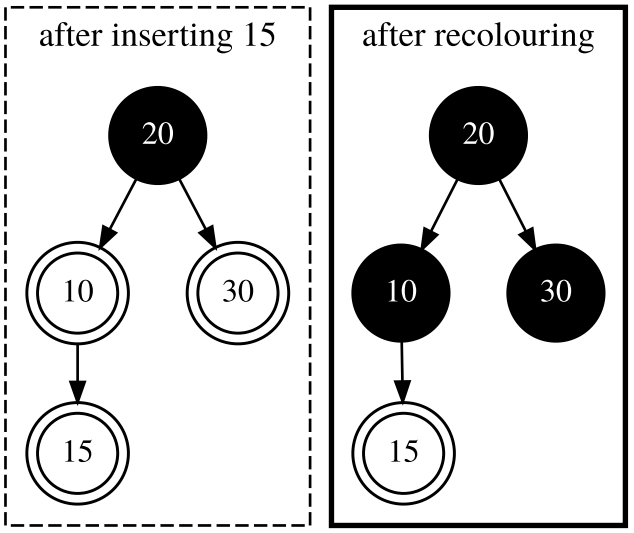
Left and right rotation (in-order sequence a, x, b, y, c is unchanged; O(1) time)

Case	Condition	Fix
1	Uncle is **RED**	Recolour: parent and uncle black, grandparent red
2	Uncle black, **triangle** (LR / RL)	Rotate at the parent (becomes Case 3)
3	Uncle black, **line** (LL / RR)	Rotate at the grandparent, swap colours

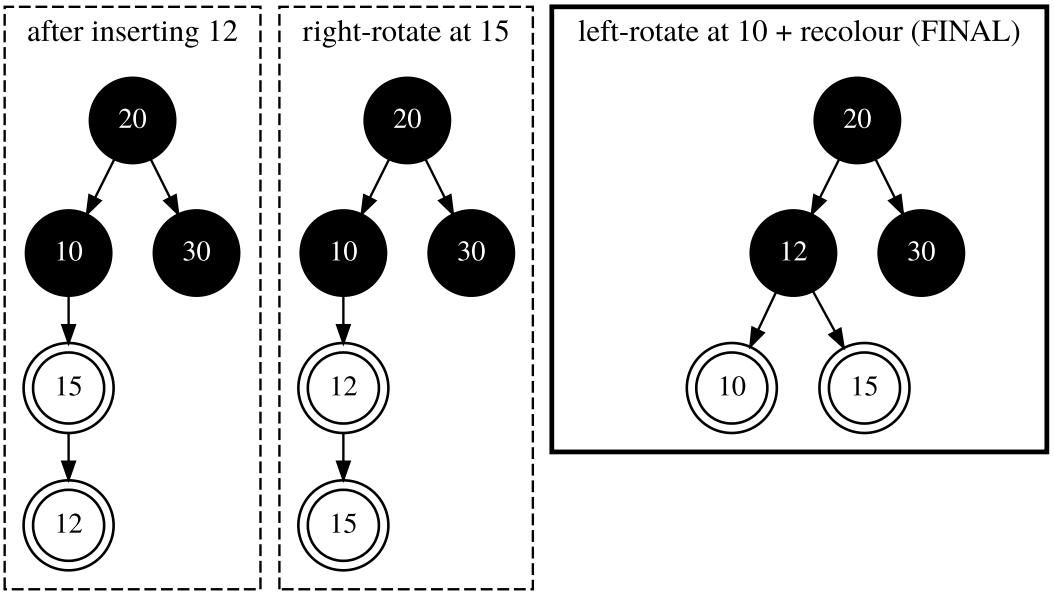
Insert 10, 20, 30, 15, 12 (filled circle = BLACK, double circle = RED)



Insert 30: Case 3 (RR line). Left-rotate at 10 and swap the colours of 10 and 20



Insert 15: Case 1 (uncle 30 is RED). Recolour 10 and 30 BLACK, 20 RED, then the root becomes BLACK again

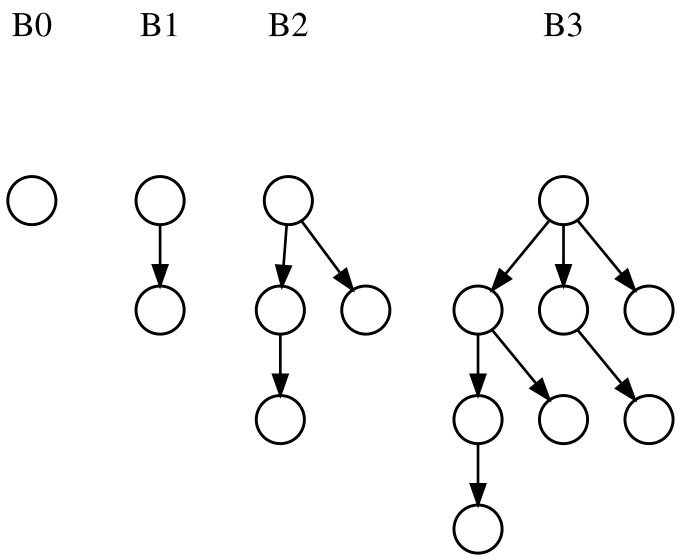


Insert 12: RL triangle. Case 2 (right-rotate at 15), then Case 3 (left-rotate at 10, recolour 12 BLACK and 10 RED)

Conclusion: at most **2 rotations per insert**. Used in Java TreeMap and C++ `std::map`.

Q5. Explain the structure, properties and union of a Binomial Heap. *[7 Marks]*

Ans. A **binomial heap** is a set of **binomial trees**. Each tree is **min-heap ordered**, and there is **at most one tree of each degree**. Its **union takes O(log n)**.



Binomial trees B0 to B3: Bk has 2^k nodes and height k; it is formed by linking two B(k−1) trees

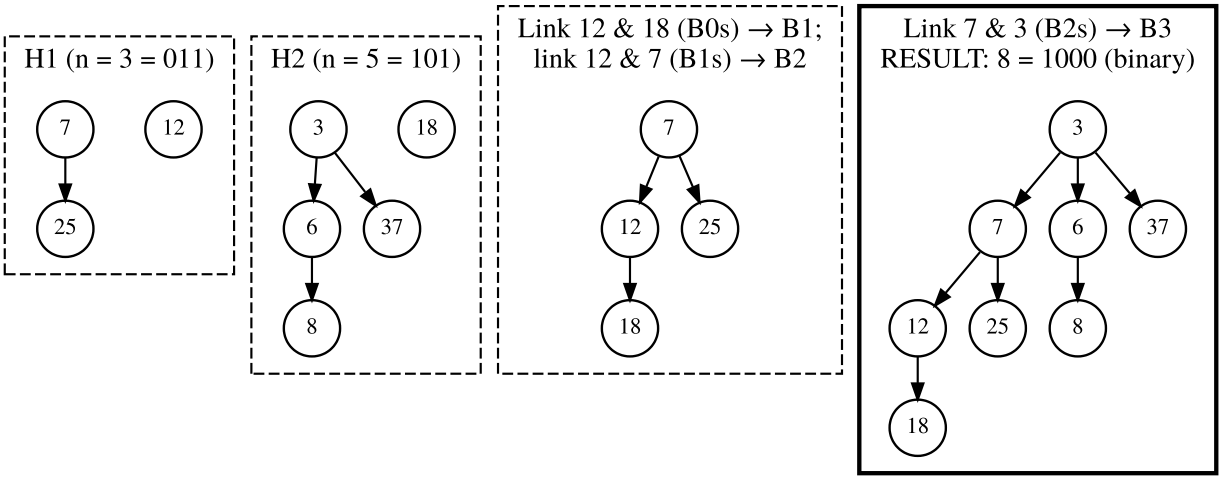
Properties of B_k:

- 2^k nodes and height k.
- $\binom{k}{i}$ nodes at depth i.
- The root has degree k.

Properties of the heap: a heap with n nodes has trees matching the **1-bits of n in binary**. For example $13 = 1101_2$ gives B_3, B_2, B_0.

Union steps:

1. **Merge** the root lists by degree.
2. **Link** trees of equal degree; the smaller key becomes the root. This works like binary addition.



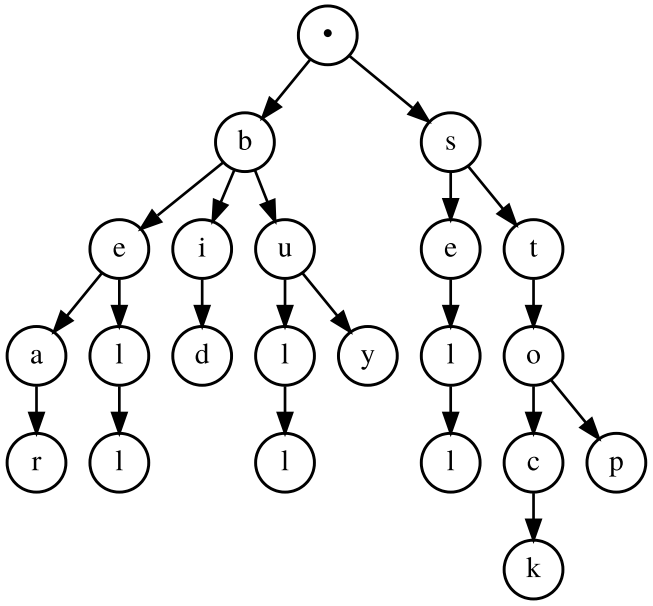
BINOMIAL-HEAP-UNION of H1 (3 nodes) and H2 (5 nodes)

Operation	Binary heap	Binomial heap
Insert	$O(\log n)$	$O(\log n)$
Extract-min	$O(\log n)$	$O(\log n)$
Union	**O(n)**	**O(log n)**

Conclusion: a mergeable priority queue, and the basis of Fibonacci heaps.

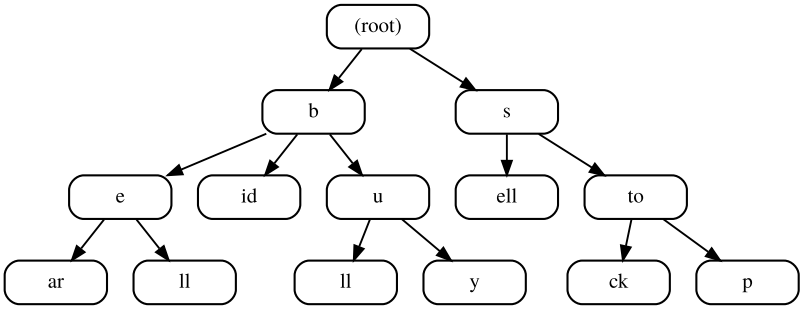
Q6. Construct a compressed trie for {bear, bell, bid, bull, buy, sell, stock, stop}. *[7 Marks]*

Ans. A **trie** stores strings character by character. A **compressed trie** merges **single-child chains into one edge**, so every internal node has **at least 2 children**.



Standard trie for {bear, bell, bid, bull, buy, sell, stock, stop} (22 nodes)

Compress: a→r becomes “ar”, i→d becomes “id”, e→l→l becomes “ell”, t→o becomes “to”, c→k becomes “ck”.



Compressed trie: single-child chains merged into one edge (14 nodes)

Standard trie	Compressed trie
Edge = 1 character	Edge = substring
22 nodes here	**14 nodes** here
O(total length) nodes	O(number of strings) nodes

Conclusion: used in autocomplete, IP routing and suffix trees.

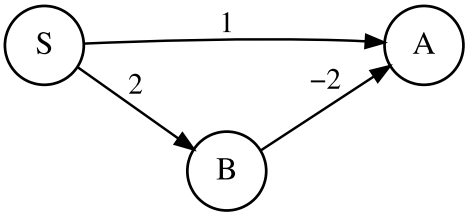
UNIT 3: GREEDY ALGORITHMS & GRAPHS

Section A: 2-Mark Questions

Q1. Greedy choice property; when Dijkstra fails. *[2 Marks]*

Ans.

- **Greedy choice property:** a **globally optimal solution can be reached by locally optimal choices**. Example: activity selection, always pick the activity that finishes earliest.
- **Dijkstra fails with negative edges**, because it finalizes a vertex too early.

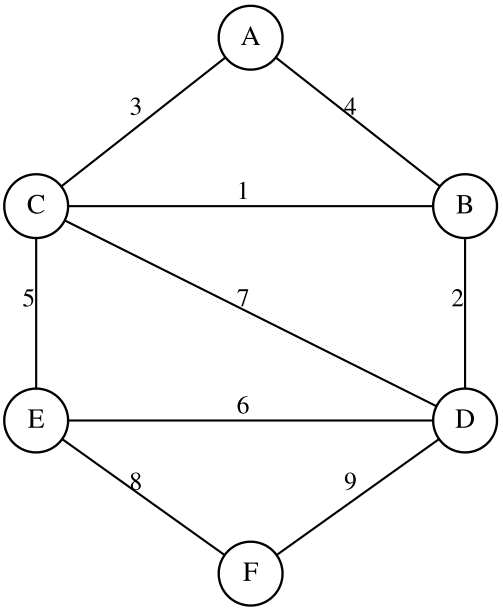


Dijkstra fails with a negative edge: it finalizes $d(A) = 1$, but the true shortest distance is $S \to B \to A = 2 + (-2) = 0$

Section B: 7-Mark Questions

Q2. Trace Kruskal's and Prim's algorithms for the MST. *[7 Marks]*

Ans. An **MST** connects all V vertices with **V – 1 edges of minimum total weight**. Kruskal and Prim are **greedy** algorithms.



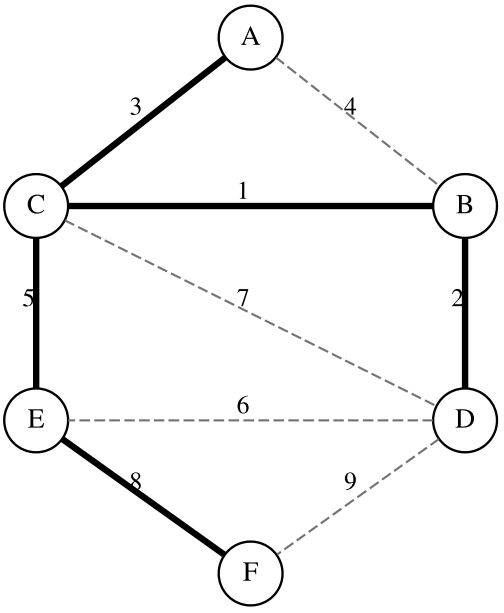
Given weighted graph (6 vertices, 9 edges)

Kruskal (sort the edges, add an edge if it makes no cycle, checked with Union-Find)

Edge	B-C 1	B-D 2	A-C 3	A-B 4	C-E 5	D-E 6	C-D 7	E-F 8
Action	✓	✓	✓	× cycle	✓	×	×	✓

Prim (from A, always add the cheapest edge leaving the tree)

Tree so far	{A}	{A,C}	{A,C,B}	{A,B,C,D}	{A,B,C,D,E}
Edge chosen	A-C 3	C-B 1	B-D 2	C-E 5	E-F 8



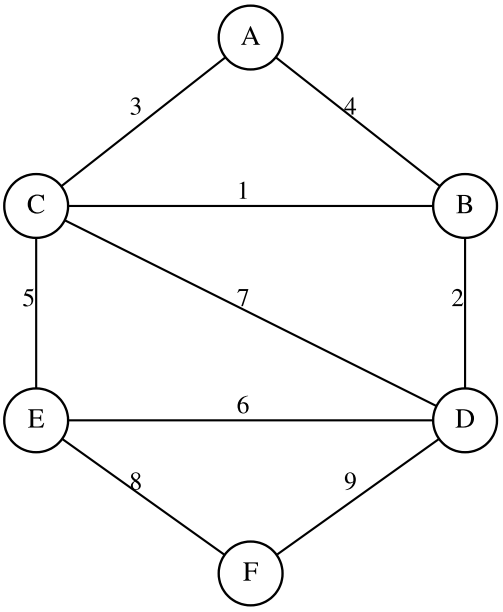
Minimum spanning tree (bold edges), total weight = 1 + 2 + 3 + 5 + 8 = 19

Kruskal	Prim
Edge-based; forest grows	Vertex-based; one tree grows
Union-Find	Priority queue
$O(E \log E)$; best for sparse graphs	$O(E \log V)$; best for dense graphs

Conclusion: both give MST weight = **19**. Used to design cable and road networks.

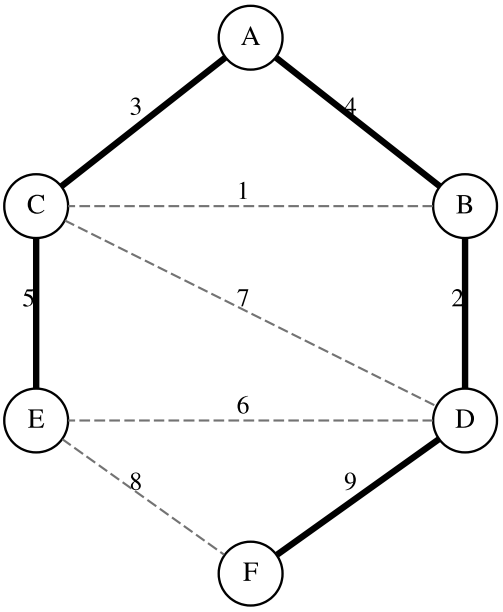
Q3. Find shortest paths using Dijkstra's algorithm. *[7 Marks]*

Ans. Dijkstra is a **greedy single-source shortest-path** algorithm for **non-negative** weights. It repeatedly picks the vertex with the minimum distance and **relaxes** its edges: if $d[u] + w(u, v) < d[v]$ then $d[v] = d[u] + w(u, v)$.



Graph used (source = A)

Picked	A	B	C	D	E	F
Init	0	∞	∞	∞	∞	∞
A (0)	**0**	4	3	∞	∞	∞
C (3)	0	4	**3**	10	8	∞
B (4)	0	**4**	3	6	8	∞
D (6)	0	4	3	**6**	8	15
E (8)	0	4	3	6	**8**	15
F (15)	0	4	3	6	8	**15**



Shortest-path tree from A (bold edges); distances A0, B4, C3, D6, E8, F15

Answer: A → B = 4, A → C = 3, A → B → D = 6, A → C → E = 8, A → B → D → F = 15.

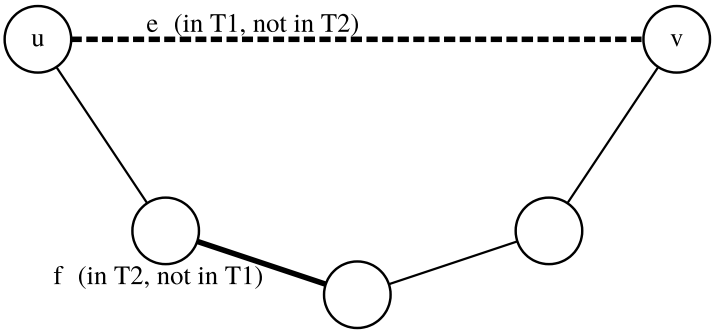
Complexity: $O\left(V^2\right)$ with an array, or $O\left(\left(V+E\right)\log V\right)$ with a heap.

Conclusion: used in GPS navigation and OSPF routing. For negative edges, use **Bellman-Ford**.

Q4. Prove that distinct edge weights give a unique MST. *[7 Marks]*

Ans. Proof by contradiction (exchange argument):

1. **Assume** there are two different MSTs, $T_1 \neq T_2$.
2. Let **e** be the **minimum-weight edge** in $T_1 \oplus T_2$ (the edges in only one of the trees). Say $e \in T_1$ and $e \notin T_2$.
3. Adding e to T_2 creates exactly **one cycle C**.
4. C must contain an edge $f \in T_2$ with $f \notin T_1$, because otherwise T_1 would contain a cycle.
5. f is in $T_1 \oplus T_2$, and the weights are distinct, so $w(e) < w(f)$.
6. $T_2' = T_2 + e - f$ is a spanning tree with $w(T_2') = w(T_2) + w(e) - w(f) < w(T_2)$. **Contradiction.**
7. **Therefore the MST is unique.** ∎



path in T2 from u to v

Exchange argument: adding e to T2 creates a cycle; removing the heavier edge f gives a lighter spanning tree

> **Remember:** the converse is false. With equal weights there may be many MSTs (for example a triangle with all weights 1 has 3).

UNIT 4: DYNAMIC PROGRAMMING

Section A: 2-Mark Questions

Q1. Principle of Optimality; Divide and Conquer vs. DP. *[2 Marks]*

Ans. Principle of optimality (Bellman): an optimal solution contains **optimal solutions to its sub-problems**.

Divide & Conquer	Dynamic Programming
Independent sub-problems	**Overlapping** sub-problems
Solves the same sub-problem again	Solves each once and **stores** it
Merge Sort, Quick Sort	Knapsack, LCS, MCM

Section B: 7-Mark Questions

Q2. Solve the 0/1 Knapsack problem with DP. W = 5, (w, v) = (2,3), (3,4), (4,5), (5,6). *[7 Marks]*

Ans.

$$V[i][w] = \begin{cases} 0 & i = 0 \text{ or } w = 0 \\ V[i-1][w] & w_i > w \\ \max\left(V[i-1][w],\ v_i + V[i-1][w-w_i]\right) & \text{otherwise} \end{cases}$$

i w	0	1	2	3	4	5
0	0	0	0	0	0	0
1 (2, 3)	0	0	3	3	3	3
2 (3, 4)	0	0	3	4	4	**7**
3 (4, 5)	0	0	3	4	5	7
4 (5, 6)	0	0	3	4	5	**7**

Sample: $V[2][5] = \max\left(V[1][5], 4 + V[1][2]\right) = \max\left(3, 7\right) = 7.$

Traceback:

- V[4][5] = V[3][5] = V[2][5], so items 4 and 3 are not taken.
- V[2][5] ≠ V[1][5], so **item 2 is taken**. Remaining capacity w = 2.
- V[1][2] ≠ V[0][2], so **item 1 is taken**.

$$\text{Items } \{1, 2\},\ \text{ maximum value} = 7,\ \ x = (1, 1, 0, 0)$$

Complexity: $O(nW)$, which is pseudo-polynomial.

Q3. Matrix Chain Multiplication: find the optimal parenthesization. p = ⟨5, 4, 6, 2, 7⟩. *[7 Marks]*

Ans.

$$m[i][j] = \min_{i \le k < j} \{m[i][k] + m[k+1][j] + p_{i-1} p_k p_j\}, \qquad m[i][i] = 0$$

L = 2:

- m[1][2] = 5·4·6 = 120
- m[2][3] = 4·6·2 = 48
- m[3][4] = 6·2·7 = 84

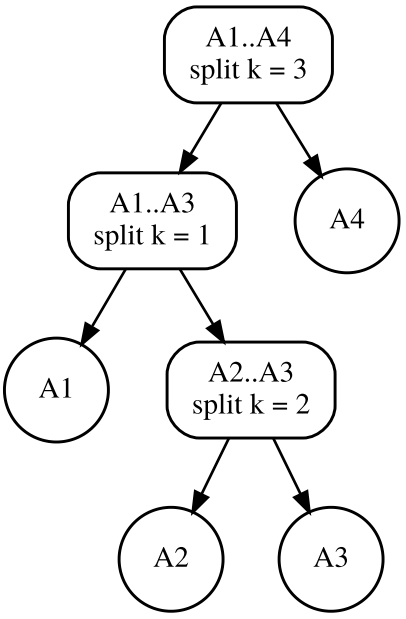
L = 3:

- m[1][3] = min(0 + 48 + 40, 120 + 0 + 60) = **88** (k = 1)
- m[2][4] = min(0 + 84 + 168, 48 + 0 + 56) = **104** (k = 3)

L = 4:

- m[1][4] = min(0 + 104 + 140, 120 + 84 + 210, 88 + 0 + 70) = **158** (k = 3)

m	1	2	3	4
1	0	120	88	**158**
2		0	48	104
3			0	84



Parenthesization tree from the s table: ((A1 (A2 A3)) A4)

$$\left(\left(A_1\left(A_2A_3\right)\right)A_4\right), \quad \text{cost} = 158$$

Complexity: $O\left(n^3\right)$ time and $O\left(n^2\right)$ space.

Q4. Find the LCS of X = ABCBDAB and Y = BDCABA. *[7 Marks]*

Ans.

$$c[i][j] = \begin{cases} 0 & i = 0 \text{ or } j = 0 \\ c[i-1][j-1] + 1 & X_i = Y_j \\ \max\left(c[i-1][j], c[i][j-1]\right) & X_i \neq Y_j \end{cases}$$

	Ø	B	D	C	A	B	A
A	0	0	0	0	1↖	1	1↖
B	0	1↖	1	1	1	2↖	2
C	0	1	1	2↖	2	2	2
B	0	1↖	1	2	2	3↖	3
D	0	1	2↖	2	2	3	3
A	0	1	2	2	3↖	3	4↖
B	0	1↖	2	2	3	4↖	**4**

(↖ marks a match, where 1 is added from the diagonal.)

Traceback from c[7][6], following the ↖ cells, picks B, C, B, A.

$$\text{LCS} = \text{BCBA}, \quad \text{length} = 4$$

Complexity: $O(mn)$.

UNIT 5: BACKTRACKING & NP

Section A: 2-Mark Questions

Q1. Backtracking; N-Queens problem. *[2 Marks]*

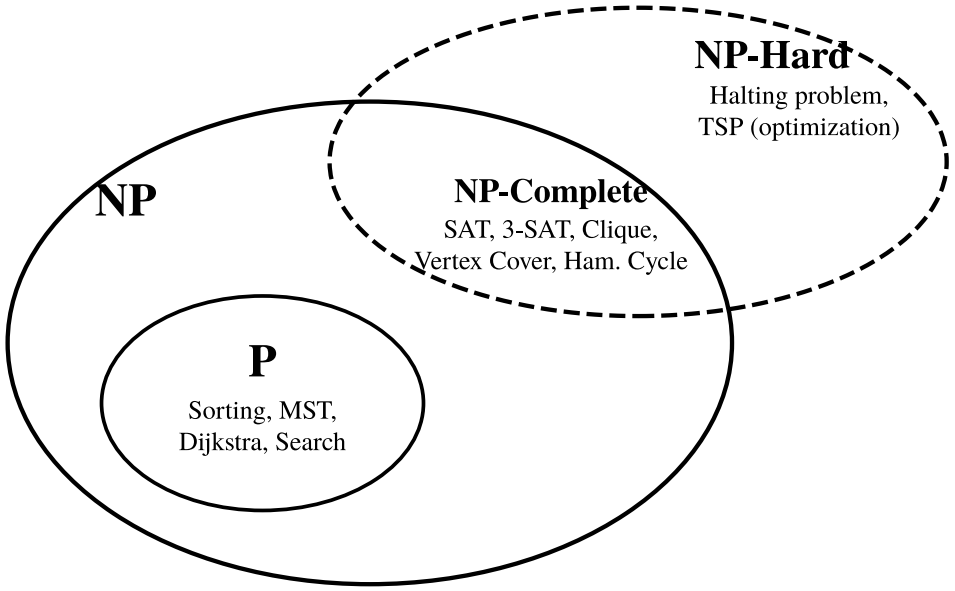
Ans.

- **Backtracking:** a **DFS of the state-space tree**. Build the solution step by step, and **undo (backtrack)** as soon as a partial solution violates a constraint.
- **N-Queens:** place N queens on an N×N board so that **no two share a row, column or diagonal**. Two queens attack if $x_i = x_j$ or $|x_i - x_j| = |i - j|$. 4-Queens has 2 solutions; 8-Queens has 92.

Section B: 7-Mark Questions

Q2. Explain P, NP, NP-Complete and NP-Hard with a diagram. *[7 Marks]*

Ans.

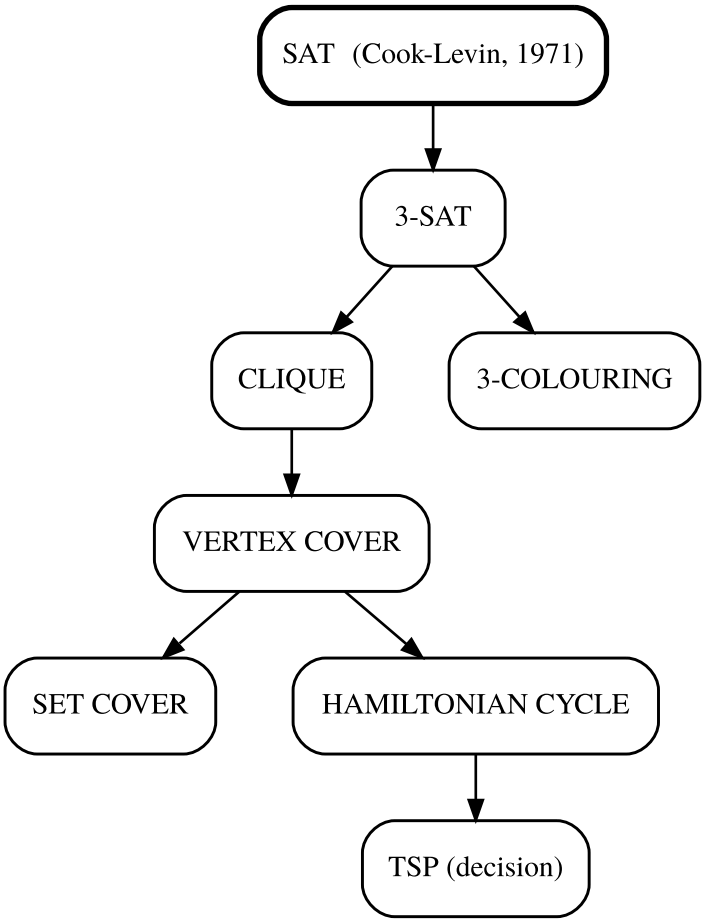


$NP\text{-}Complete = NP \cap NP\text{-}Hard; \quad P \subseteq NP$

Relationship between P, NP, NP-Complete and NP-Hard (assuming $P \neq NP$)

1. **P:** problems **solvable** in polynomial time. Examples: sorting, MST, Dijkstra.
2. **NP:** problems whose solutions are **verifiable** in polynomial time. **P ⊆ NP.** Example: Sudoku.
3. **NP-Hard: every NP problem reduces to it** ($L \leq_p H$). It **need not be in NP**. Example: the Halting problem.
4. **NP-Complete = NP ∩ NP-Hard.** Examples: **SAT (Cook-Levin, the first)**, 3-SAT, Clique, Vertex Cover.

5. **To prove X is NP-Complete:** (i) show $X \in NP$, and (ii) reduce a known NPC problem to X.
6. If **any NPC problem is solved in polynomial time, then P = NP**. This question is still unsolved.



Chain of polynomial-time reductions between NP-Complete problems

	P	NP	NPC	NP-Hard
Solvable in poly time	Yes	?	?	?
Verifiable in poly time	Yes	Yes	Yes	Not necessarily
In NP?	Yes	Yes	Yes	Not necessarily

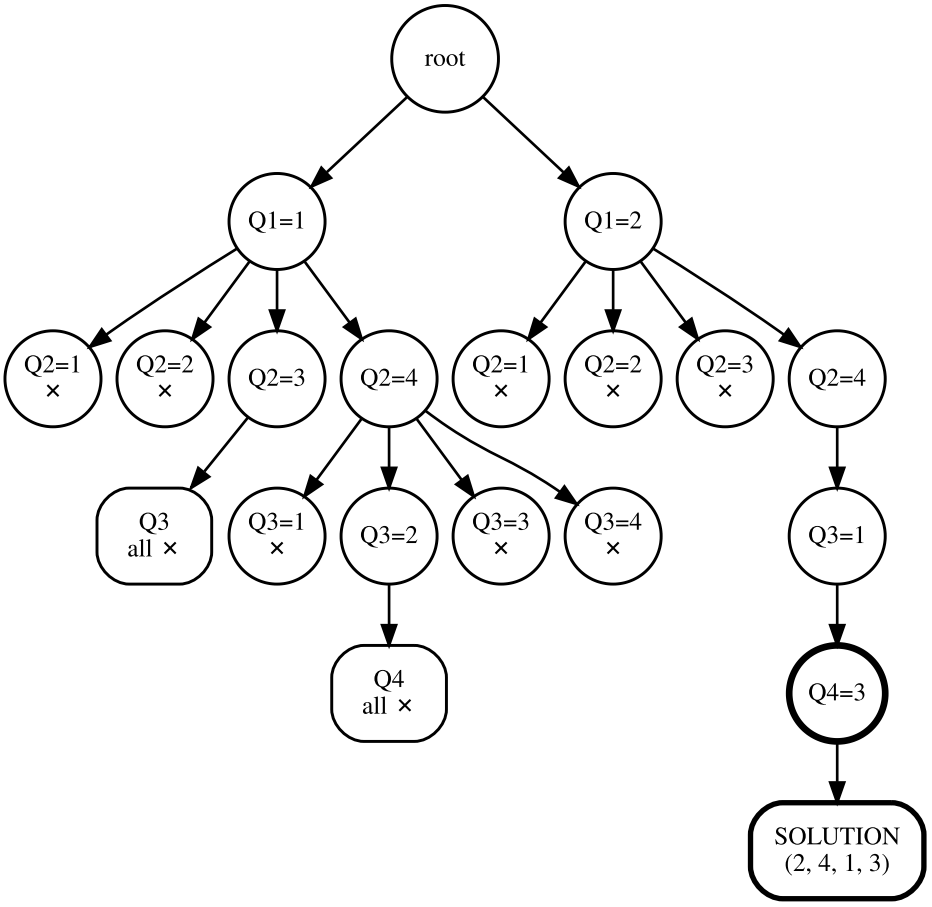
Conclusion: for NPC problems (TSP, scheduling) we use **approximation and heuristics**.

Q3. Solve the 4-Queens problem using backtracking. *[7 Marks]*

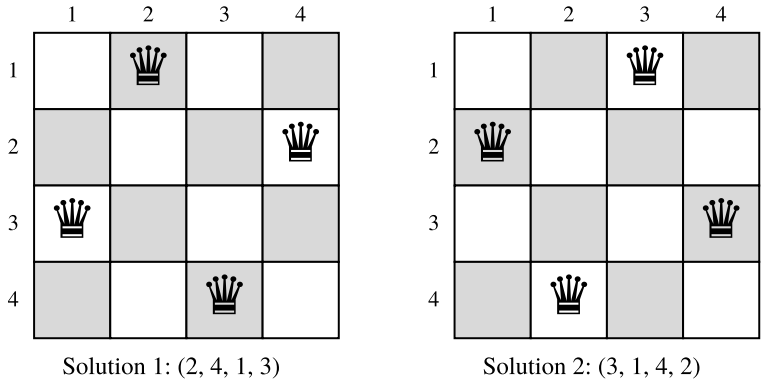
Ans. Place the queens **row by row**. For each row, try columns 1 to 4. **PLACE(k, i)** checks the column and the diagonals, and the algorithm **backtracks** when no column is safe.

```
NQUEENS(k, n): for i = 1 to n:
    if PLACE(k, i): x[k] = i;  if k == n: print x  else: NQUEENS(k+1, n)
PLACE(k, i): for j = 1 to k-1:  if x[j] == i or |x[j]-i| == |j-k|: return false
             return true
```

Step	Try	Result
1	Q1 = 1, Q2 = 3	Q3 has no safe column, so **backtrack**
2	Q2 = 4, Q3 = 2	Q4 has no safe column, so **backtrack** to Q1
3	Q1 = 2, Q2 = 4, Q3 = 1, Q4 = 3	**Solution**



Pruned state-space tree for 4-Queens (× = killed by the bounding function)



The two solutions of 4-Queens (Solution 2 is the mirror image of Solution 1)

Answer: (2, 4, 1, 3) and its mirror (3, 1, 4, 2). **8-Queens:** 92 solutions; the first found is (1, 5, 8, 6, 3, 7, 2, 4).

Conclusion: backtracking prunes the search space. It is used in Sudoku solvers and timetabling.